Low-Latency Concept Drift Detection and Online Adaptation for High-Throughput Streaming Classification

Yash Patidar
School of Computer Engineering
Manipal Institute of Technology,
Manipal Academy of Higher Education
Manipal, Karnataka
yash8.mitmpl2024@learner.manipal.edu

Pragun Kakkar
School of Computer Engineering
Manipal Institute of Technology,
Manipal Academy of Higher Education
Manipal, Karnataka
pragun.mitmpl2024@learner.manipal.edu

Shantanu Singh
School of Computer Engineering
Manipal Institute of Technology,
Manipal Academy of Higher Education
Manipal, Karnataka
shantanu.mitmpl2024@learner.manipal.edu

***Abstract*—Data streams from sensor networks and transaction systems change over time. When the data distribution shifts, a model trained on older data loses accuracy. This is called concept drift. Many drift detectors are either expensive to run or use fixed thresholds that raise false alarms when the data changes slowly. We built a drift detector that costs $O(1)$ time and memory per sample and keeps no window of past data. It is a one-sided Page-Hinkley test on the classifier's binary error signal, measured against a bias-corrected Exponentially Weighted Moving Average (EWMA) of past errors. We placed it in a test-then-train (prequential) pipeline with two incremental classifiers from `river`. The first is a Hoeffding Adaptive Tree (HAT), which adapts through its own ADWIN-based subtree replacement. The second is a streaming Logistic Regression (LR) pipeline, which we reset on every alarm. On a synthetic SEA stream with an abrupt change at sample 25,000, the detector raised one alarm per model, 1,213 samples after the change for HAT and 1,356 for LR, and none before the change. LR reached 99.3% overall prequential accuracy against 97.1% for HAT. Its rolling accuracy stayed at or above 0.95 from 1,837 samples after the change, compared with 6,519 for HAT. On the real-world Elec2 benchmark, each model raised 10 alarms and the two reached almost the same overall accuracy (83.7% for LR, 84.0% for HAT). But LR took 10.61 μs per sample on average and HAT took 77.32 μs, so LR ran 7.3 times faster (6.2 times on SEA). A linear model with a constant-time detector and hard resets matched the adaptive tree's accuracy on Elec2 and beat it on SEA, at a fraction of the cost.**

***Index Terms*—concept drift, online learning, data stream mining, EWMA, Page-Hinkley test, Hoeffding Adaptive Tree, prequential evaluation**

I. Introduction and Motivation

Many systems process data that never stops arriving: sensor networks, trading platforms, and network intrusion monitors are common examples. The process that generates this data changes as the environment, user behaviour, or the economy changes. This is concept drift [1]. A model trained on past data loses accuracy when it happens [3], so a streaming model has to keep learning from each new instance.

Streaming also limits how much time and memory a model can spend on each record [4]. Recent drift detectors built on deep learning and learned representations [4], [7] can find complex shifts. But they need large models, add latency, and buffer data, which does not fit streams that must be processed in microseconds [4]. Simple statistical detectors are cheap, but many use fixed thresholds that raise false alarms when the data shifts slowly [6]. And reactive detectors only fire after accuracy has already been falling for a while [8].

We test a middle ground. Our detector is a Page-Hinkley cumulative-sum test that compares each error with a bias-corrected EWMA of past errors. It costs $O(1)$ time and memory per sample. It follows slow changes in the error rate and still reacts to sustained increases. We pair it with two incremental classifiers, a Hoeffding Adaptive Tree and a streaming Logistic Regression pipeline, and evaluate both with the prequential protocol on one synthetic and one real-world benchmark.

Our contributions are:

- An $O(1)$-time, $O(1)$-memory Page-Hinkley drift detector with a bias-corrected EWMA baseline. We wrote it from first principles, without `river.drift`, and it keeps no window of past data.
- A prequential pipeline with a separate adaptation hook for each model. HAT relies on its own ADWIN-driven subtree replacement, and our alarm does not change it. LR gets a hard reset of both the scaler and the model.
- An evaluation on SEA and Elec2 that reports rolling and overall accuracy, detection delay, recovery, and per-sample wall-clock latency measured with `time.perf_counter_ns`.
- A check of every reported result against the exported telemetry. We rebuild the per-sample error sequence from the logged rolling accuracy, and replaying the detector on it reproduces all 22 logged alarm positions.

Section II reviews related work. Section III states the research gap, Section IV the problem, and Section V the objectives. Section VI describes the datasets, Section VII the method, and Section VIII the implementation. Section IX gives the results, and Section X discusses them. Section XI concludes and outlines future work.

II. Literature Survey and Comparative Analysis

Work on drift detection trades computational cost against sensitivity. Early statistical tests were cheap but did not adapt well to gradual shifts. More recent work uses deep networks and representation learning. Wan et al. [4] detect drift without labels by computing a Maximum Concept Discrepancy (MCD) with contrastive learning, which works in high-dimensional spaces. Chambers et al. [7] built AdaDeepStream, which uses intermediate CNN activations to detect drift with few true labels. These methods work well on complex data, but their cost creates latency bottlenecks on fast streams.

Other work reduces the need for labels or offline tuning. Fellicious et al. [10] proposed SUDS, an unsupervised drift sampling method that picks homogeneous samples for retraining and cuts annotation needs by up to 80%. Wu and Xiong [2] update the parameters of Bayesian classifiers online. Their method is cheap but sensitive to class imbalance. Aguiar and Cano [5] use meta-learning on sliding-window meta-features to pick the best drift detector, which requires extensive offline meta-training. Mulimani et al. [1] combine ensembles with adaptive sliding windows in an online LightGBM framework. It handles gradual drift well but reacts slowly to abrupt changes.

To cut cost, some recent work has gone back to statistical and probabilistic methods. Parasteh and Sadaoui [3] proposed PRDD, which tracks shifts in classifier confidence in $O(1)$ time. It shows that monitoring the prediction error stream can be sensitive without buffering raw data. Rum and Song [6] showed that updating reconstruction-error thresholds with an EWMA baseline greatly reduces false alarms on stable and gradually drifting streams. Proactive methods try to adapt before accuracy drops. Guerrero Cano et al. [8] built Hoeffding Tree variants that estimate where the class boundary is moving, and Zhao and Shen [9] proposed PROCEED, which adjusts model parameters ahead of drift in online time-series forecasting. Table I compares these approaches.

III. Research Gap

Two gaps stand out in current work on streaming drift adaptation:

1) **Cost versus fixed thresholds:** Deep and representation-based detectors [4], [7] find complex drifts but are too slow and memory-hungry for fast streams. Lightweight statistical detectors often use fixed thresholds that do not adjust to slow drift, so they raise frequent false alarms [6].
2) **Slow reaction:** Most lightweight stream methods react only after accuracy has dropped far enough to give statistical evidence of drift [8]. Few constant-time frameworks adjust their own baseline to catch drift quickly while still processing each sample in microseconds.

IV. Problem Statement

We aim to design, build, and evaluate a low-latency drift detection and online adaptation framework for fast data streams. It must detect distribution shifts in a streaming classifier and trigger adaptation as they happen. It must keep the accuracy loss after a drift small, and its detector must run in $O(1)$ time and memory per sample.

V. Project Objectives

The project has four objectives:

- **Objective 1:** Build a constant-time $O(1)$ drift detector based on the Page-Hinkley cumulative-sum test that tracks the classification error stream without sliding windows [3].
- **Objective 2:** Give the detector an adaptive baseline, a bias-corrected Exponentially Weighted Moving Average (EWMA) of the error, that keeps recalibrating the expected error level and suppresses false alarms caused by noise [6].
- **Objective 3:** Connect the detector to incremental learners (a Hoeffding Adaptive Tree and streaming Logistic Regression) so that drift alarms trigger online adaptation [8].
- **Objective 4:** Evaluate the whole framework with the prequential protocol on synthetic and real-world streams, measuring detection delay, recovery, accuracy, and per-sample processing time [3], [6].

VI. Dataset Description and Preprocessing

We use two standard benchmarks: a synthetic stream with one controlled, abrupt change, and a real-world stream with natural drift whose timing is unknown.

A. Synthetic SEA Concept Generator

We use the SEA generator of Street and Kim through `river.datasets.synth.SEA`. Each instance has three attributes f_1, f_2, f_3, drawn uniformly from $[0, 10]$. The label is $y =$ True if $f_1 + f_2 > \theta$ and False otherwise. The attribute f_3 does not affect the label. Label noise is off (`noise=0`). The stream uses `variant=0` ($\theta = 8$) before the change and `variant=1` ($\theta = 9$) after it, both with seed 42. The classes are not balanced: 68.2% of the first 25,000 instances are positive, and so are 59.8% of the last 25,000. The change only relabels instances with $8 < f_1 + f_2 \leq 9$, which make up 8.4% of the post-change instances. So a model that kept the old boundary perfectly would see its error rate rise by about 8.4 percentage points.

The function `concept_drift_stream()` in `river_benchmarks.py` mixes the two concepts. It copies the sigmoid-weighted mixing of `river`'s `ConceptDriftStream` but computes the sigmoid in a numerically stable way. The library computes $1/(1 + e^v)$ directly, which overflows for a narrow width far from the change point. At sample i, the next instance comes from the new concept with probability $\sigma(4(i-p)/w)$, where $p = 25{,}000$ and $w = 1$. That probability is 0.018 at $i = 24{,}999$, 0.5 at $i = 25{,}000$, and 0.982 at $i = 25{,}001$, so the change is effectively instant. In the generated stream, samples 1–25,000 come from the first concept and samples 25,001–50,000 from the second. Both generators use seed 42, so the second half repeats the feature vectors of the first half, relabelled with $\theta = 9$. We cut the stream at $N = 50{,}000$ instances and process it in order.

TABLE I
COMPARATIVE ANALYSIS OF RECENT CONCEPT DRIFT DETECTION METHODOLOGIES

Year	Title & Authors	Proposed Methodology	Strengths	Limitations
2023	*Online Learning of Bayesian Classifiers...* [2]	Updates Bayesian parameters online, one sample at a time [2].	Cheap to compute and handles non-stationary streams [2].	No mechanism for imbalanced multi-class data [2].
2023	*AdaDeepStream: Streaming Adaptation...* [7]	Feeds CNN activations to streaming models to detect drift [7].	Adapts quickly with few true labels [7].	Large architectural overhead and needs explicit drift detection [7].
2023	*Enhancing Concept Drift Detection...* [5]	Uses sliding-window meta-features to recommend a drift detector [5].	Handles class imbalance without committing to one detector [5].	Needs offline training of meta-models on large datasets [5].
2024	*Online Detection and Adaptation...* [1]	Online LightGBM that combines ensembles with adaptive sliding windows [1].	Adapts to both real and virtual drift [1].	Struggles with sudden, abrupt drift [1].
2024	*A Probabilistic Approach for...* [3]	PRDD tracks shifts in posterior probability with adaptive confidence [3].	$O(1)$ time per sample [3].	Depends on how well the base classifier's probabilities are calibrated [3].
2024	*Online Drift Detection with...* [4]	Computes Maximum Concept Discrepancy (MCD) with contrastive learning [4].	Needs no labels and works on complex, high-dimensional data [4].	Training the deep encoder takes significant resources [4].
2024	*SUDS: A Strategy for Unsupervised...* [10]	Samples homogeneous data for retraining without supervision [10].	Cuts the labelled data needed by up to 80% [10].	Underperforms on artificial datasets without noise [10].
2025	*PROCEED: Proactive Model Adaptation...* [9]	Maps latent drift to parameter changes ahead of time to get around feedback delays [9].	Anticipates drift before performance drops [9].	Relies on past drift patterns and struggles with out-of-distribution shifts [9].
2026	*An EWMA-Based Adaptive...* [6]	Updates autoencoder reconstruction-error thresholds with an EWMA [6].	Removes false alarms on stable and gradually drifting streams [6].	Sensitive to the EWMA smoothing coefficient [6].
2026	*Anticipating to Change...* [8]	Hoeffding Tree variants that anticipate drift by estimating boundary movement [8].	Prevents degradation early by maximizing class separability [8].	Assumes linear boundary movement and struggles with erratic drift [8].

B. Electricity Market Benchmark (Elec2)

Elec2 comes from the Australian New South Wales (NSW) electricity market. Harries compiled it in 1999, and we load it with `river.datasets.Elec2`. It has 45,312 instances recorded every 30 minutes from May 1996 to December 1998. Each instance has eight attributes: date, day of week (1–7), half-hour period, NSW price, NSW demand, Victoria (VIC) price, VIC demand, and the scheduled NSW–VIC transfer. In the version `river` uses, every attribute except day of week is already min–max normalized to $[0, 1]$. The label says whether the NSW price went UP (42.5%) or DOWN (57.5%) compared with its average over the previous 24 hours. `river` maps UP to True. Elec2 drifts naturally with demand, regulation, and the seasons, but nobody knows exactly when. We process the full stream in time order, with no filtering or reordering.

C. Preprocessing and Feature Engineering

All preprocessing happens online:

- **Streaming standardization (LR only):** A `river.preprocessing.StandardScaler` keeps running estimates of each feature's mean and variance and rescales each instance to roughly zero mean and unit variance.
- **No scaling for HAT:** The tree uses raw attribute values. Its splits depend only on the order of the values, so rescaling would not change them.
- **Labels:** Both loaders already return boolean labels (SEA: $f_1 + f_2 > \theta$, Elec2: UP), so we apply no further encoding.
- **No feature engineering:** We add no interaction terms, lag features, or polynomial terms. Both models see exactly the attributes the loaders return, including Elec2's date. Any difference between the models therefore comes from the learners and how they adapt, not from the features.

VII. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

A. System Architecture and Pipeline Workflow

The pipeline is the prequential loop in `evaluate()` (`stream_evaluation.py`), shown in Fig. 1. For each labelled instance (x_t, y_t) it runs five steps in this order:

1) **Predict:** The current model outputs $\hat{y}_t = f(x_t)$ before it sees y_t.
2) **Metrics and error:** We update the rolling accuracy and cumulative F1 score with $(y_t, \hat{y}_t)$ and compute the binary error $e_t = \mathbf{1}[\hat{y}_t \neq y_t]$.
3) **Learn:** The model trains on (x_t, y_t) through $f.\text{LEARN}(x_t, y_t)$. The latency timer, started before step 1, stops here.
4) **Monitor:** The EWMA-PH detector takes e_t and updates its state in $O(1)$ operations.
5) **Adapt:** If $PH_t > \lambda$, we record the alarm at t, call the model's adaptation function $\text{ADAPT}(f)$, and reset the detector. Learning happens before monitoring, so the old model has already learned the instance that triggered the alarm. A reset model starts learning from x_{t+1}.

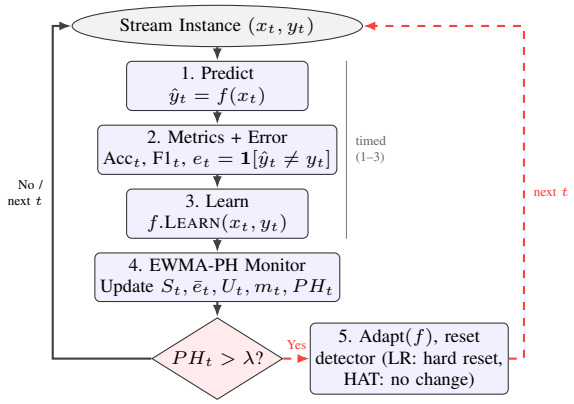


Fig. 1. The prequential pipeline in `evaluate()`. The latency timer covers steps 1–3. The detector update and adaptation (steps 4–5) run after the model has learned (x_t, y_t).

B. Adaptive Drift Detection via EWMA-PH

The drift monitor, `PageHinkleyDetector` in `page_hinkley.py`, is a Page-Hinkley cumulative-sum test. Its reference level is an exponentially weighted running mean of the errors, and we wrote it from first principles. It takes the error signal $e_t \in \{0, 1\}$ and updates three quantities.

1. Bias-Corrected EWMA Baseline: An accumulator S_t tracks the error rate:

$$S_t = \alpha\, S_{t-1} + (1 - \alpha)\, e_t, \quad S_0 = 0 \tag{1}$$

where $\alpha \in (0, 1]$ is the fading factor. A raw exponential average starts out biased towards zero, so we correct it, updating α^t as we go:

$$\bar{e}_t = \frac{S_t}{1 - \alpha^t} \tag{2}$$

While $t \ll 1/(1 - \alpha)$, $\bar{e}_t$ is close to the plain average of all errors so far. As t grows, $\alpha^t \to 0$ and $\bar{e}_t$ becomes an exponential moving average that remembers about $1/(1 - \alpha)$ samples. So the baseline follows slow changes in the error rate, and a few early errors do not dominate it [6].

2. Cumulative-Sum Statistic: U_t adds up how far each error lies above the baseline, minus a tolerance δ:

$$U_t = U_{t-1} + \big(e_t - \bar{e}_t - \delta\big), \quad U_0 = 0 \tag{3}$$

When nothing changes, $\mathbb{E}[e_t - \bar{e}_t - \delta] \approx -\delta < 0$. U_t then behaves like a random walk that drifts slightly downwards, and noise does not build up into an alarm.

3. Test Statistic and Drift Decision: The detector tracks the running minimum of U_t,

$$m_t = \min(m_{t-1}, U_t), \quad m_0 = 0, \tag{4}$$

and the Page-Hinkley statistic measures how far U_t has risen above it:

$$PH_t = U_t - m_t. \tag{5}$$

When the error rate jumps, $\bar{e}_t$ lags behind. The increments $(e_t - \bar{e}_t - \delta)$ stay positive, and U_t climbs away from its minimum. The detector signals drift as soon as

$$PH_t > \lambda. \tag{6}$$

The test is one-sided. It only reacts when errors increase, which is the direction that hurts the model. If the error rate jumps from a baseline $\bar{e}$ to p, the statistic grows by about $p - \bar{e} - \delta$ per sample, so detection takes roughly $\lambda/(p - \bar{e} - \delta)$ samples. After an alarm, `reset()` sets $n, S_t, \bar{e}_t, U_t, m_t$, and PH_t back to 0 and α^t back to 1, so monitoring starts over with a fresh baseline. There is no warm-up period. Each update takes a fixed number of arithmetic operations. The detector stores seven state variables $(n, S_t, \alpha^t, \bar{e}_t, U_t, m_t, PH_t)$ and three hyperparameters $(\alpha, \delta, \lambda)$.

C. Model Selection Justification

We compare two incremental classifiers, both with `river`'s default hyperparameters:

- **Hoeffding Adaptive Tree (HAT):** `river`'s `HoeffdingAdaptiveTreeClassifier` chooses splits with Hoeffding bounds (grace period 200, split confidence 10^{-7}) and watches every node with an ADWIN detector. When a node's error rises, it grows an alternate subtree and swaps it in if the new subtree becomes clearly more accurate. So HAT adapts on its own and can model non-linear boundaries. It uses bootstrap sampling with no fixed seed by default, so two runs do not give identical results.
- **Streaming Logistic Regression (LR):** a `river` pipeline of `StandardScaler` followed by `LogisticRegression`, trained with SGD (learning rate 0.01, log loss, no regularization). It costs $O(d)$ per sample for d features. It has no built-in way to forget old data, and it is fully deterministic.

The pair lets us compare a high-capacity model with a fast one, and internal adaptation with detector-triggered adaptation. We use an EWMA reference level instead of the plain cumulative mean of the classic Page-Hinkley test, because a cumulative mean reacts less and less as t grows [3]. Sliding-window methods avoid that problem but must store past observations [1]. The EWMA baseline adjusts its reference level in $O(1)$ space and time [6].

D. Online Adaptation Protocol

When an alarm fires ($PH_t > \lambda$), the pipeline calls the model's adaptation function:

- **Logistic Regression pipeline** (`reset_logreg_pipeline`): The SGD weights and the scaler's running statistics hold on to the old concept indefinitely. So we replace both pipeline steps with new `StandardScaler` and `LogisticRegression` instances. We call this a *hard reset*.
- **Hoeffding Adaptive Tree** (`reset_hat`): This function does nothing, on purpose. HAT keeps its structure and adapts only through its own ADWIN-driven subtree replacement. For HAT, our detector only monitors, and HAT serves as the reference for internal adaptation.

E. Evaluation Metrics Formulation

We compute the following for each run:

- **Rolling accuracy (Acc_t):** accuracy over the last $W = 500$ predictions (`utils.Rolling(metrics.Accuracy, 500)`), logged after every sample: $\text{Acc}_t = \frac{1}{\min(t,W)} \sum_{i=\max(1,t-W+1)}^{t} \mathbf{1}[\hat{y}_i = y_i]$.
- **Final rolling accuracy:** Acc_N, the accuracy over the last 500 samples. It covers only 500 predictions, so it depends heavily on what the stream is doing at its end.
- **Overall prequential accuracy:** $\frac{1}{N} \sum_{t=1}^{N} \mathbf{1}[\hat{y}_t = y_t]$, the accuracy over the whole stream.
- **F1 score:** `river.metrics.F1`, the cumulative binary F1 score of the positive class (True: $f_1 + f_2 > \theta$ for SEA, UP for Elec2).
- **Detection delay (D):** $D = t_{\text{alarm}} - t^*$ for the injected change at $t^* = 25{,}000$ (SEA only).
- **Recovery (SEA only):** the lowest rolling accuracy after the change, the first sample after that low point where $\text{Acc}_t \geq 0.95$ again, and the point after which Acc_t never drops below 0.95 again (sustained recovery). We report each as an offset from t^*.
- **Per-sample latency:** wall-clock time measured with `time.perf_counter_ns` around prediction, the two metric updates, and learning (steps 1–3). We report the mean, the median, and the 95th and 99th percentiles in microseconds. The detector update, the adaptation call, and logging fall outside the timed region.
- **Alarm count (K):** the number of drift alarms in the stream.

Algorithm 1 summarizes the procedure.

Algorithm 1 EWMA-PH Prequential Evaluation and Online Adaptation

Input: Data stream $\mathcal{S}$, model f, detector parameters $(\alpha, \delta, \lambda)$, adaptation function ADAPT

Detector state: $n \leftarrow 0,\ a \leftarrow 1,\ S \leftarrow 0,\ U \leftarrow 0,\ m \leftarrow 0$
for each instance $(x_t, y_t) \in \mathcal{S}$ **do**
  $t_{\text{start}} \leftarrow$ PERFCOUNTERNS()
  $\hat{y}_t \leftarrow f$.PREDICT(x_t)
  Update Acc_t (window 500) and F1 with $(y_t, \hat{y}_t)$
  $e_t \leftarrow \mathbf{1}[\hat{y}_t \neq y_t]$
  f.LEARN(x_t, y_t)
  $\Delta t_t \leftarrow$ PERFCOUNTERNS() $- t_{\text{start}}$
  $n \leftarrow n + 1,\ a \leftarrow \alpha\, a\ \{a = \alpha^n\}$
  $S \leftarrow \alpha S + (1 - \alpha) e_t,\ \bar{e} \leftarrow S/(1 - a)$
  $U \leftarrow U + (e_t - \bar{e} - \delta),\ m \leftarrow \min(m, U),\ PH \leftarrow U - m$
  if $PH > \lambda$ **then**
    Record alarm at t, then call ADAPT(f) {LR: hard reset, HAT: no change}
    Reset detector: $n, S, \bar{e}, U, m, PH \leftarrow 0$ and $a \leftarrow 1$
  end if
  Log $(t, \text{Acc}_t, \Delta t_t, \text{alarm flag})$
end for

VIII. Implementation Details

A. Software

We wrote the framework in Python 3 on top of `river`, which supplies the streaming learners, metrics, and dataset loaders. We draw the figures with `matplotlib` (`seaborn-v0_8-paper` style). We write and read telemetry with Python's standard `csv` module. The project code does not use `numpy` or `pandas` directly. `page_hinkley.py` implements the detector from first principles, without `river.drift`.

B. Repository Structure

The repository (`Project_FML`) contains these scripts:

- `page_hinkley.py`: the $O(1)$ EWMA-PH detector (1)–(6), plus a self-test on a synthetic Bernoulli error stream whose rate jumps from 0.05 to 0.45.
- `stream_evaluation.py`: the prequential `evaluate()` loop (rolling accuracy, F1, per-sample latency, drift monitoring, adaptation hooks) and the two adaptation functions.
- `river_benchmarks.py`: the benchmark driver. It builds the SEA stream with the stable mixing generator, loads Elec2, runs both models, and writes per-sample telemetry to `telemetry_sea.csv` and `telemetry_elec2.csv` (columns: sample index, model, rolling accuracy, latency in μs, drift flag).
- `plot_results.py`: draws the recovery curve (`figure1_recovery.png`) and the latency–accuracy plot (`figure2_pareto.png`) from the telemetry.
- `analyze_telemetry.py`: recomputes the derived metrics in Section IX from the telemetry (see Section VIII-F).
- `stream_dataset.py`, `make_sample.py`: a generator that streams a CSV file row by row and converts numeric fields, and a script that writes a 20-row random CSV (`sample_stream.csv`). Running `stream_evaluation.py` directly on that file is a smoke test of the pipeline (`telemetry_results.csv`). It is not part of the reported results.

C. Execution Environment

We produced all reported runs and timings on the authors' Apple-silicon Mac running macOS, in one Python process, evaluating the models one after the other. Every logged latency is a whole multiple of 41.67 ns, the resolution of the 24 MHz system timer on Apple-silicon machines. That is more than 100 times finer than the per-sample latencies we measure. Absolute latencies depend on the CPU and the Python runtime, so the ratios between models carry over to other machines better than the raw numbers do.

D. Hyperparameter Settings

- **EWMA-PH detector:** fading factor $\alpha = 0.9999$ (a memory of $1/(1 - \alpha) = 10{,}000$ samples), tolerance $\delta = 0.005$, and threshold $\lambda = 50$. These are the standard Page-Hinkley settings and the defaults of `river`'s own `PageHinkley`. We fixed them in advance and did not tune them on either stream. Each model and dataset gets a fresh detector.
- **Evaluation:** rolling window $W = 500$. Telemetry is written for every sample, and a progress line is printed every 100 samples.
- **SEA stream:** variants 0 ($\theta = 8$) and 1 ($\theta = 9$), seed 42 for both generators and for the mixing, change at position 25,000, width 1, and $N = 50{,}000$.

- **Classifiers:** `river` defaults for `HoeffdingAdaptiveTreeClassifier` (no seed) and for `LogisticRegression` (SGD, learning rate 0.01).

E. Training, Validation, and Testing Procedure

k-fold cross-validation and fixed train/test splits make no sense for a stream that changes over time. We use the prequential protocol instead. Each instance first tests the current model and only then trains it, so every prediction is made on unseen data, in a single pass. We used no separate validation split and tuned no hyperparameters on the evaluation streams.

F. Telemetry Verification and Reproducibility

Because we know the window size, we can recover from the logged rolling accuracy whether each prediction was correct ($c_t = \min(t, W)\,\mathrm{Acc}_t - \sum_{i=\max(1,t-W+1)}^{t-1} c_i$). `analyze_telemetry.py` does this for all four runs. It then recomputes overall accuracy, F1 (using the dataset labels), and the recovery metrics, and replays the detector offline on $e_t = 1 - c_t$. The replay reproduces all 22 logged alarms (1 + 1 on SEA, 10 + 10 on Elec2) at exactly the logged sample indices. So the telemetry and the detector code agree.

The SEA stream and the LR pipeline are deterministic. A separate re-run of the SEA benchmark (`river` 0.26.1, Python 3.13, Linux) reproduced LR's rolling accuracy exactly at all 50,000 samples, including the alarm at sample 26,356. HAT has no seed, so its results change from run to run. In five re-runs it again raised exactly one alarm, with delays from 1,056 to 1,356 samples and overall accuracy from 0.967 to 0.971 (0.971 in the reported run). We do not use latencies from these re-runs anywhere in this paper.

IX. Experimental Results and Performance Evaluation

Tables II and III summarize accuracy, detection, and latency for the four runs. Table IV covers the SEA change in detail.

TABLE II
Accuracy and Drift Detection

Dataset	Model	Overall Acc. (all N)	Final Acc. (last 500)	F1	Alarms	Delay
SEA (50k)	HAT	0.971	0.988	0.978	1	1,213
SEA (50k)	Scaler+LR	0.993	0.998	0.994	1	1,356
Elec2 (45k)	HAT	0.840	0.794	0.807	10	N/A
Elec2 (45k)	Scaler+LR	0.837	0.912	0.799	10	N/A

TABLE III
Per-Sample Latency in μs (Predict + Metric Updates + Learn)

Dataset	Model	Mean	Median	p95	p99	Samples/s
SEA	HAT	44.40	39.83	61.71	103.54	22,500
SEA	Scaler+LR	7.15	7.04	7.33	9.04	139,900
Elec2	HAT	77.32	63.50	167.00	320.83	12,900
Elec2	Scaler+LR	10.61	10.50	11.08	12.92	94,300

Samples/s $= 10^6/$mean. This leaves out the detector update and loop overhead, so it is an upper bound on end-to-end throughput.

TABLE IV
SEA Concept Change ($t^* = 25{,}000$): Detection and Recovery

	HAT	Scaler+LR
Accuracy, samples 1–25,000	0.971	0.994
Max. PH_t, samples 1–25,000 ($\lambda = 50$)	20.8	6.0
Error rate between change and alarm	0.074	0.049
Detection delay D	1,213	1,356
Lowest rolling accuracy (sample)	0.890 (26,770)	0.934 (25,508)
First return to $\mathrm{Acc}_t \geq 0.95$	$t^* + 2{,}823$	$t^* + 953$
Sustained $\mathrm{Acc}_t \geq 0.95$ from	$t^* + 6{,}519$	$t^* + 1{,}837$
Accuracy, samples 25,001–50,000	0.971	0.992

A. Evaluation on the Synthetic SEA Stream

Before the change, LR's rolling accuracy approaches 1.0 after a short warm-up, with 0.994 accuracy over samples 1–25,000. HAT is lower and noisier at 0.971, and its rolling accuracy falls as low as 0.936 after warm-up. Neither model gets a false alarm before the change. The largest Page-Hinkley statistic before sample 25,000 is 20.8 for HAT and 6.0 for LR, well under $\lambda = 50$. Here is what happens after the change at sample 25,000 (Fig. 2, Table IV):

- **Detection:** Each model gets exactly one alarm, HAT at sample 26,213 (delay 1,213) and LR at sample 26,356 (delay 1,356). No other alarm follows. The delays fit the estimate $\lambda/(p - \bar{e} - \delta)$. Between the change and the alarm, the error rate was 7.4% for HAT and 4.9% for LR, about 4 to 5 percentage points above each model's error rate before the change. LR's error rose less, and its alarm came later, because SGD had already started moving its boundary towards the new concept.
- **Adaptation before the alarm:** Both post-change error rates are below the 8.4% that a model stuck on the old boundary would make. So both learners were already adapting before any alarm. LR's rolling accuracy got back to 0.95 at $t^* + 953$, *before* its alarm at $t^* + 1{,}356$.
- **Effect of the LR hard reset:** The reset therefore threw away a model that had mostly re-adapted. Accuracy over the 500 samples after the reset was 0.952, against 0.970 over the 500 samples before it. It then rose to 0.968 and 0.983 over the next 500 and 1,000 samples. LR's rolling accuracy stays at or above 0.95 from $t^* + 1{,}837$ (481 samples after the reset) and ends the stream at 0.998.
- **HAT recovery:** HAT's alarm does not change the model. HAT recovers through ADWIN-driven subtree replacement and further tree growth. Its rolling accuracy drops to 0.890 at sample 26,770, first gets back to 0.95 at $t^* + 2{,}823$, and stays above 0.95 only from $t^* + 6{,}519$. It ends the stream at 0.988.
- **Overall:** LR's overall prequential accuracy is 0.993 (F1 0.994) and HAT's is 0.971 (F1 0.978). LR is more accurate both before and after the change.
- **Latency:** LR takes 7.15 μs per sample on average (median 7.04 μs), 6.2 times faster than HAT's 44.40 μs (median 39.83 μs). LR's latency is also far more consistent (p99 of 9.04 against 103.54 μs).

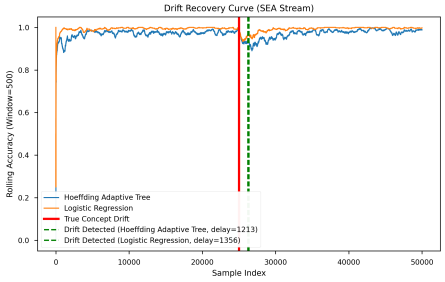


Fig. 2. Rolling accuracy ($W = 500$) on the SEA stream. The red line marks the concept change at sample 25,000. The dashed green lines mark the detector alarms for HAT (sample 26,213, delay 1,213) and LR (sample 26,356, delay 1,356). LR is hard-reset at its alarm. HAT's alarm does not change it.

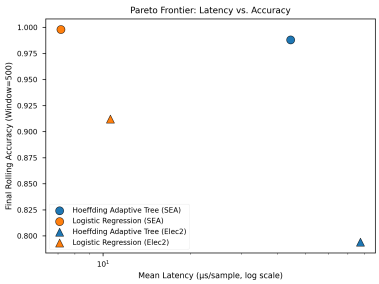


Fig. 3. Mean per-sample latency (log scale) against final rolling accuracy (last 500 samples), from `plot_results.py`. Circles show SEA and triangles show Elec2. Orange is Scaler+LR and blue is HAT. Table II gives overall prequential accuracy.

B. Evaluation on the Electricity Market Benchmark (Elec2)

Each model raises 10 alarms on Elec2 (45,312 samples, Table II).

- **Alarm positions:** LR: 2,472, 5,352, 15,689, 20,524, 22,782, 27,389, 31,366, 36,606, 43,334, 44,032. HAT: 6,236, 9,431, 14,961, 16,227, 19,327, 22,834, 24,522, 31,136, 36,548, 43,793. The positions differ because each detector watches its own model's errors. The gaps between consecutive alarms range from 698 to 10,337 samples for LR (median 4,607) and from 1,266 to 7,245 for HAT (median 3,507). Elec2 has no ground-truth drift points, so we cannot say which alarms are true and which are false.
- **Accuracy:** Over the whole stream the two models are effectively tied. Overall prequential accuracy is 0.840 for HAT and 0.837 for LR, with F1 scores for the UP class of 0.807 and 0.799. Across nine 5,000-sample blocks, LR wins five and HAT wins four. LR's rolling accuracy is above HAT's at 50.0% of the samples. Both curves swing widely (HAT 0.63–0.98, LR 0.67–0.95).
- **Final rolling accuracy:** The last 500-sample window favours LR (0.912 against 0.794). This reflects the end of the stream, not the whole run. LR's last alarm at sample 44,032 reset it 1,280 samples before the end, and over samples 44,033–45,312 LR scored 0.876 against 0.802 for HAT.
- **Latency:** LR takes 10.61 μs per sample on average (median 10.50 μs), 7.3 times faster than HAT's 77.32 μs (median 63.50 μs). HAT has a long tail (p99 of 320.83 μs). Both models are slower than on SEA, probably because Elec2 has eight attributes instead of three. For HAT, the size of the tree and the number of ADWIN monitors also matter.

C. Latency–Accuracy Trade-off

Fig. 3, produced by `plot_results.py`, plots mean latency (log scale) against final rolling accuracy. On SEA, LR beats HAT on both axes, whichever accuracy measure is used (Table II). On Elec2, the accuracy gap in Fig. 3 exists only in the last 500 samples. In overall accuracy the two models differ by 0.2 percentage points. So on Elec2, LR's advantage is speed (7.3 times faster) at the same accuracy, not higher accuracy.

D. Comparison with Baseline Models

HAT is our built-in adaptive baseline. It adapts with `river`'s ADWIN-based alternate subtrees, and the external detector does not change it. The Scaler+LR pipeline represents explicit detection followed by a hard reset. The repository has no runs of other baselines, such as LR without resets, a Page-Hinkley test without the EWMA baseline, or other detectors like DDM or ADWIN driving LR. So we cannot yet separate how much the EWMA baseline and the reset each contribute. Adding these runs only takes new entries in `make_model_configs()` in `river_benchmarks.py`, for example an LR adaptation function that does nothing, or a different detector object.

X. Discussion and Result Analysis

A. Interpretation of Results

1) **Elec2 accuracy matches the original estimate:** The project synopsis expected both models to reach about 83.6% on Elec2. The overall accuracies, 83.7% for LR and 84.0% for HAT, agree. The large gap in final rolling accuracy (91.2% against 79.4%) comes from one 500-sample window. It does not show that LR beats HAT on Elec2.
2) **What the resets do and do not show:** Accuracy often rebounds after an alarm. But HAT rebounds too, even though its alarm leaves the model unchanged. Comparing the 500 samples before and after each alarm, HAT went from 0.738 to 0.844 around sample 22,834 and from 0.806 to 0.908 around sample 14,961. Alarms tend to fire at local low points, so a rebound after an alarm does not prove that the reset helped. On SEA, LR had mostly re-adapted through SGD before its alarm, and the reset caused a short, small dip before accuracy rose above its pre-reset level. Measuring the real benefit of resets needs the no-reset run described above.
3) **Linear model versus tree:** SEA's concepts are linear in (f_1, f_2), so a linear model fits them almost exactly. HAT has to approximate the diagonal boundary with axis-parallel splits. That explains why LR is more accurate before and after the change. Elec2's boundary is not linear, and there the tree's extra capacity and LR's resets end up at the same overall accuracy.

4) **Latency:** LR is 6.2 times faster than HAT on SEA and 7.3 times faster on Elec2. A linear prediction and SGD update need one $O(d)$ dot product and one vector update. HAT has to walk the tree, update split statistics at the leaf, check Hoeffding bounds, and update the ADWIN monitors along the path.
5) **Detection:** On SEA, the detector raised no false alarm in 25,000 stable samples per model and one alarm after the change. That alarm came 1,213 (HAT) and 1,356 (LR) samples after an error increase of 4 to 5 percentage points. Larger jumps in error get detected proportionally faster. On Elec2, the detector raised about one alarm every 4,500 samples per model, so it did not fire constantly.
6) **Context on Elec2:** A trivial classifier that just repeats the previous label scores 85.3% on Elec2, because the labels are strongly autocorrelated [11]. Both models score slightly below it, so their absolute Elec2 accuracies need to be read with care.

B. Strengths

- **Constant detection cost:** Each detector update is a fixed number of arithmetic operations on seven state variables, and the detector stores no past observations.
- **High throughput:** The Scaler+LR pipeline handles roughly 94,000 to 140,000 predict-and-learn cycles per second on one core (Table III). That is 6 to 7 times more than HAT, with very consistent latency.
- **No false alarms on the stable part of SEA:** The largest statistic before the change was 6.0 to 20.8 against a threshold of 50, and the detector still caught the abrupt change for both models.
- **Verifiable results:** Every alarm reproduces exactly from the logged telemetry, and the LR run reproduces exactly in an independent re-run on different hardware.

C. Limitations

- **Detection delay:** With $\lambda = 50$, a moderate error increase of 4 to 5 percentage points takes more than 1,200 samples to detect. On SEA, LR had already recovered through SGD before its alarm.
- **Missing ablations:** We have no runs of LR without resets, of a plain (non-EWMA) Page-Hinkley test, or of other detectors. So we cannot isolate what the EWMA baseline and the hard reset each contribute.
- **Hard resets lose information:** A reset discards every learned weight and the scaler's statistics. This causes a cold-start dip (seen on SEA) and makes false alarms expensive.
- **Few benchmarks:** We used two binary streams, one with a single abrupt change. We have not tested gradual, recurring, multi-class, or high-dimensional streams, and Elec2 has no ground-truth drift points.
- **One run, unseeded HAT:** We ran each configuration once. HAT has no seed, so its alarm positions and accuracy vary between runs (Section VIII-F).
- **Sensitivity to α, δ, and λ:** We used the standard defaults without tuning. Streams that drift often may need other values [6].
- **What we measured:** The latencies leave out the detector update and loop overhead and come from one machine. We did not profile memory use, such as HAT's tree growth.

D. Connection to Research Gap and Objectives

- **Objective 1 (constant-time detector):** Met. The detector runs in $O(1)$ time and memory on the error stream with no windows, and replaying it reproduces all logged alarms.
- **Objective 2 (adaptive EWMA baseline):** Met in implementation. With the bias-corrected EWMA baseline, the detector raised no false alarm on the stable part of SEA and a timely alarm after the change. We have no fixed-baseline Page-Hinkley run, so we have not yet measured how much of this comes from the EWMA baseline.
- **Objective 3 (coupling with incremental learners):** Met. Both learners run inside the prequential pipeline. LR gets a hard reset on every alarm. HAT uses its own adaptation, and the detector only monitors it.
- **Objective 4 (prequential evaluation):** Met. We measured detection delay (1,213 and 1,356 samples), recovery (sustained ≥ 0.95 from $t^* + 1{,}837$ for LR and $t^* + 6{,}519$ for HAT), accuracy (overall 0.993 and 0.971 on SEA, 0.837 and 0.840 on Elec2), and latency (7.15 and 44.40 μs on SEA, 10.61 and 77.32 μs on Elec2).

XI. Conclusion and Future Scope

We built and evaluated a drift detection and online adaptation framework for streaming classification. At its core is a Page-Hinkley test with a bias-corrected EWMA baseline that runs in $O(1)$ time and memory per sample. On the SEA stream with an abrupt change, the detector raised exactly one alarm per model, 1,213 samples after the change for HAT and 1,356 for LR, and none in the stable part. The streaming LR pipeline reached 99.3% overall accuracy and stayed at or above 0.95 rolling accuracy from 1,837 samples after the change. HAT reached 97.1% and took 6,519 samples. On Elec2, LR with hard resets matched HAT's overall accuracy (83.7% against 84.0%) and processed each sample 7.3 times faster (10.61 against 77.32 μs). On these streams, a fast linear model with a constant-time drift detector competes with an adaptive tree at a much lower cost. We still need to show what the hard reset adds over plain incremental learning.

Next steps:

1) **Ablation baselines:** LR without resets, a fixed-baseline Page-Hinkley test, and ADWIN- or DDM-triggered adaptation, each run with several seeds.
2) **Softer adaptation:** weight decay, warm-started replacements, or ensemble blending instead of hard resets, to avoid cold-start dips during gradual drift.
3) **Faster detection:** tuning α, δ, and λ automatically [6], and predicting drift before it happens [8], [9].

4) **More benchmarks:** gradual and recurring synthetic drift with known change points, and multi-class and high-dimensional streams.
5) **Resource profiling:** end-to-end throughput including the detector, and memory use of the learners.

USE OF AI TOOLS

In line with academic integrity guidelines, we disclose that we used AI tools while preparing this report. We used them for LaTeX typesetting, checking the report against the evaluation rubric, grammar and style edits, and auditing the manuscript against the repository's code and telemetry. That audit included drafting `analyze_telemetry.py`, which recomputes the derived metrics from the logged telemetry. The authors wrote the framework, ran the benchmarks, and produced the telemetry, and every reported number can be regenerated from the repository.

REFERENCES

[1] D. Mulimani, P. Patil, S. Totad, and R. Benni, "Online Detection and Adaptation of Concept Drift in Streaming Data Classification," *Procedia Computer Science*, vol. 235, pp. 2803–2811, 2024.

[2] P. Wu and N. Xiong, "Online Learning of Bayesian Classifiers with Nonstationary Data Streams," *International Journal of Network Dynamics and Intelligence*, vol. 2, no. 3, 2023.

[3] S. Parasteh and S. Sadaoui, "A Probabilistic Approach for Detecting Real Concept Drift," *Proceedings of the 16th International Conference on Agents and Artificial Intelligence (ICAART)*, vol. 2, pp. 301–311, 2024.

[4] K. Wan, Y. Liang, and S. Yoon, "Online Drift Detection with Maximum Concept Discrepancy," *Proceedings of the 30th ACM SIGKDD Conference*, pp. 2924–2935, 2024.

[5] G. J. Aguiar and A. Cano, "Enhancing Concept Drift Detection in Drifting and Imbalanced Data Streams through Meta-Learning," *Virginia Commonwealth University*, 2023.

[6] S. N. Mohd Rum and Q. Song, "An EWMA-Based Adaptive Thresholding Concept for Autoencoder-Based Concept Drift Detection in Data Streams," *International Journal of Advanced Computer Science and Applications*, vol. 17, no. 5, 2026.

[7] L. Chambers, M. M. Gaber, and H. Ghomeshi, "AdaDeepStream: streaming adaptation to concept evolution in deep neural networks," *Applied Intelligence*, vol. 53, pp. 27323–27343, 2023.

[8] J. V. Guerrero Cano, G. J. Aguiar, and A. Cano, "Anticipating to Change: A Proactive Approach for Concept Drift Adaptation in Data Streams," *Machine Learning*, vol. 115, no. 3, 2026.

[9] L. Zhao and Y. Shen, "Proactive Model Adaptation Against Concept Drift for Online Time Series Forecasting," *Proceedings of the 31st ACM SIGKDD Conference*, 2025.

[10] C. Fellicious, J. Mitrović, L. Wendlinger, M. Gancarski, and M. Granitzer, "SUDS: A Strategy for Unsupervised Drift Sampling," *arXiv preprint arXiv:2411.02995*, 2024.

[11] I. Žliobaitė, "How good is the Electricity benchmark for evaluating concept drift adaptation," *arXiv preprint arXiv:1301.3524*, 2013.